

DOSSIER D'ANLAYSE ET DE CONCEPTION DU PROJET EN SYSTEMES D'EXPLOITATION

THEME :

Réimplémentation de la fonction Watch sous Linux

Membres du groupe 5 :

- ♦ SELIHE Emmanuel (**Chef**)
- ♦ TSAFAC TEUFAC Belvira Lins
- ♦ NDJOMO MELINGUI Emilienne Christelle
- ♦ BOULI ONANDA Grégory Kévin
- ♦ BELLA Gisèle Grace

Etudiants d'ingé 3 SRT

Sous la supervision de : M NGUIMBUS Emmanuel

2025-2026

Table des matières

INTRODUCTION	3
I. Analyse des besoins	4
1. Les fonctionnalités attendues ou besoins fonctionnels	4
a. <i>Les fonctionnalités principales ou vitales</i>	4
b. <i>Les fonctionnalités optionnelles avancées</i>	4
2. Les contraintes techniques et environnementales ou besoins non fonctionnels	5
a. <i>Les contraintes d'environnement et de portabilité (Linux/POSIX)</i>	5
b. <i>Les contraintes d'indépendance vis-à-vis de la bibliothèque standard (appels systèmes directs)</i>	6
c. <i>Les contraintes de gestion stricte des ressources et de la mémoire</i>	6
d. <i>Les contraintes d'interface et d'affichage (performance visuelle)</i>	6
II. Architecture et Conception Technique (Le "Comment")	7
1. Architecture globale	7
2. Logique algorithmique et structures de données	7
3. Découpage modulaire (les fonctions clés)	7
III. Interfaces et maquettes	8
1. Interface de la ligne de commande	8
2. Visuel du Terminal	9
3. Conception technique du module d'affichage	9
IV. Plan des tests de validation	10
CONCLUSION	11

INTRODUCTION

Afin de mettre en pratique les connaissances acquises dans l'unité d'enseignement *théorie des systèmes d'exploitation* et dans le cadre de la réalisation des projets de fin d'année liée à cette unité d'enseignement il nous a été demandé de réimplémenter la commande standard **watch** de Linux. Nous avons appelé notre commande clone **mywatch** et celle-ci a été entièrement programmée en langage C. L'enjeu majeur de ce projet est de s'affranchir des abstractions de haut niveau et de la réimplémenter entièrement et communiquer directement avec le noyau de Linux en implémentant les appels systèmes. Dans cette perspective, le développement de mywatch a exigé une approche rigoureuse axée sur la programmation système de bas niveau. L'atteinte de cet objectif a nécessité la décomposition de l'application en plusieurs modules interconnectés, permettant notamment une gestion logicielle autonome et sécurisée. Ainsi, l'analyse de la ligne de commande a été entièrement conçue manuellement, s'affranchissant des fonctions standard comme `getopt` pour trier les arguments directement depuis le vecteur `argv`. De surcroît, afin de garantir une surveillance réactive et robuste, le programme exploite l'appel système (`syscall`) pour intercepter les flux d'entrée en temps réel et permettre un contrôle interactif au clavier, tout en manipulant les processus via des mécanismes de duplication et de redirection de descripteurs de fichiers. Ce dossier d'analyse et de conception a donc pour vocation de formaliser l'architecture modulaire retenue, de détailler les structures de données implémentées par notre équipe, et de justifier les choix techniques qui sous-tendent notre interaction directe avec le noyau Linux.

I. Analyse des besoins

1. Les fonctionnalités attendues ou besoins fonctionnels

Dans le cadre de cette réimplémentation, les fonctionnalités de mywatch ont été segmentées et hiérarchisées en deux grandes catégories :

a. Les fonctionnalités principales ou vitales

Elles constituent le socle technique indispensable à l'émulation de la commande système, le noyau de l'application. Sans leur exécution conjointe, le programme ne peut répondre au besoin primaire de surveillance.

- **L'ordonnancement et boucle d'exécution périodique** : l'application doit maintenir une routine d'exécution infinie capable de scander l'appel aux commandes système à un intervalle temporel strict (par défaut **2.0** secondes). Cette régularité est assurée par la fonction `precise_sleep`. Contrairement à un `sleep()` standard limité à l'arrondi supérieur, le programme isole informatiquement la partie entière et la partie décimale du temps choisi par l'utilisateur pour alimenter un appel système `nanosleep()` (via la structure `struct timespec`), garantissant une précision à l'échelle de la nanoseconde.
- **Isolation de l'exécution et capture des flux à bas niveau (`execute_cmd`)** : l'outil doit être capable de lancer n'importe quelle commande fournie, de manière isolée, sans bloquer ou corrompre le processus superviseur (mywatch). Le programme réalise un clonage de processus via l'appel système `fork()` (appel direct `syscall(57)`). Le processus fils redirige dynamiquement ses descripteurs de sortie standard (`stdout`) et d'erreur standard (`stderr`) à l'aide de `dup2` (via `syscall(33)`) à l'intérieur d'un canal de communication IPC (pipe ou `syscall(22)`). Le processus père se charge ensuite de lire ce flux par blocs de 4096 octets pour centraliser le résultat textuel en mémoire vive.
- **Gestion de l'affichage plein écran et rafraîchissement d'interface (`display_init`)** : Le programme doit restituer le résultat textuel en mode plein écran sans provoquer de défilement (scrolling) vertical continu du terminal. Dès l'initialisation, l'application bascule sur l'écran alternatif du terminal en transmettant la séquence d'échappement ANSI. Le curseur clignotant est masqué. À chaque cycle applicatif, l'écran est intégralement réinitialisé et le curseur repositionné à l'origine via une commande ANSI `\033[2J\033[H`, simulant un rafraîchissement d'interface graphique en mode console.
- **Interception des signaux et terminaison sécurisée (`setup_signals`)** : L'application doit impérativement capturer les interruptions utilisateur (comme un `Ctrl+C`) pour éviter les fuites de processus (processus zombies) et restaurer l'environnement de l'utilisateur. Le programme déroute le signal `SIGINT` vers une routine de nettoyage (`quit`). Lors de l'interruption, cette fonction libère les buffers de mémoire alloués, réactive l'affichage du curseur, désactive l'écran alternatif pour restituer au terminal son état d'origine, puis ferme proprement l'application (`exit(0)`).

b. Les fonctionnalités optionnelles avancées

Elles enrichissent l'expérience utilisateur et étendent les capacités opérationnelles de l'outil. Elles représentent l'ensemble des commutateurs (flags) et options avancées configurables par l'utilisateur (via la ligne de commande) pour affiner le comportement de la surveillance. Ces besoins sont :

- **Analyse comparative et surlignage vidéo inversé (-d / args.highlight) :** Offrir une aide visuelle immédiate en mettant en exergue les modifications textuelles survenues entre le cycle actuel qu'on va appeler **t** et le cycle précédent qu'on va appeler **t-1**. La fonction `display_output` compare l'adresse et le contenu de `current_output` avec `previous_output` octet par octet. En cas de non-concordance, le caractère modifié est encapsulé dynamiquement entre les séquences de formatage ANSI `\033[7m` (inversion vidéo de la couleur) et `\033[0m` (restauration du style standard).
- **Masquage structurel du bloc d'en-tête (-t / args.no_title) :** Permettre l'optimisation de l'espace d'affichage utile en masquant les métadonnées système (horloge, intervalle, libellé). Un branchement conditionnel dans la boucle principale court-circuite l'appel à la fonction `display_header()`. L'écran applique directement l'effacement ANSI standard et dédie 100 % de la grille du terminal à l'affichage de la commande surveillée.
- **Notification sonore sur anomalie système (-b / args.beep) :** Alerter l'opérateur en tâche de fond dès que la commande observée renvoie un code d'erreur. À la fin de chaque exécution de processus, le programme intercepte le statut de sortie via la macro `WEXITSTATUS`. Si `exit_status` est différent de 0, le programme pousse le caractère d'échappement ANSI `\a` (ASCII Bell) vers le flux standard de sortie, ce qui déclenche un bip sonore matériel au niveau du système de l'hôte.
- **Filtrage et neutralisation des couleurs ANSI natives (-C / args.color_mode = 0) :** Forcer l'affichage en texte brut (Raw Text) d'une commande qui génère nativement ses propres styles colorés, afin d'éviter les surcharges visuelles. Le programme invoque le module `strip_ansi_colors` sur le pointeur de données textuelles récupéré. Un algorithme de balayage détecte les motifs types `\033[` jusqu'au caractère terminal de modification `m` pour réajuster les pointeurs de chaînes (src et dst) et écraser ces séquences avant leur affichage effectif.
- **Interruption sur rupture d'exécution (-e / args.exit_error) :** Automatiser l'arrêt de la surveillance si le service ou la commande ciblée subit un crash ou un dysfonctionnement. Dès que le code de retour intercepté (`exit_status`) est anormal (différent de 0), le programme brise immédiatement la structure itérative (`break`), nettoie les allocations dynamiques en cours et quitte le programme prématurément pour redonner la main à l'utilisateur.

2. Les contraintes techniques et environnementales ou besoins non fonctionnels

L'implémentation des fonctionnalités de `mywatch2` doit respecter un ensemble de contraintes architecturales et de bas niveau imposées par l'environnement système Linux et les choix de conception de l'équipe.

a. Les contraintes d'environnement et de portabilité (Linux/POSIX)

- **Description :** Le programme est exclusivement conçu pour s'exécuter dans un environnement de type Unix (Linux, macOS ou WSL sous Windows).
- **Traduction dans le code :** L'application s'appuie massivement sur des fichiers d'en-tête de l'interface POSIX (`<unistd.h>`, `<sys/wait.h>`, `<signal.h>`). Le script de test automatique utilise des commandes typiques du shell Bash (`timeout`, `date +%s%N`), ce qui restreint l'exécution et la validation de l'outil à un terminal Linux natif.

b. Les contraintes d'indépendance vis-à-vis de la bibliothèque standard (appels systèmes directs)

- **Description :** Pour des raisons d'optimisation, de sécurité et de maîtrise logicielle, le programme doit minimiser son utilisation des fonctions enveloppes de la bibliothèque C standard (glibc) pour les opérations critiques.
- **Traduction dans le code :** C'est l'une des contraintes les plus visibles de votre projet. Dans `executor.c` et `display.c`, les fonctions classiques sont remplacées par l'architecture des syscalls directs (via `<sys/syscall.h>`) :
 - La création de processus n'utilise pas `fork()` mais `syscall(57)`.
 - La redirection de flux n'utilise pas `dup2()` mais `syscall(33)`.
 - La lecture et la fermeture des descripteurs passent par `syscall(0)` (`read`) et `syscall(3)` (`close`).
 - La récupération des dimensions du terminal passe par `syscall(16)` (`ioctl` avec le drapeau `TIOCGWINSZ`).

c. Les contraintes de gestion stricte des ressources et de la mémoire

- **Description :** S'agissant d'un utilitaire de surveillance destiné à tourner potentiellement pendant des heures ou des jours (processus démon/long-running), le programme ne doit tolérer aucune fuite de mémoire (memory leak) ni accumulation de processus fantômes (processus zombies).
- **Traduction dans le code :**
 - **Mémoire :** Chaque allocation dynamique (`malloc` ou `realloc` dans `executor.c` pour accumuler par blocs de 4096 octets la sortie de la commande) est rigoureusement vérifiée. En cas d'erreur ou à la fermeture (`display_cleanup`), les pointeurs `current_output` et `previous_output` sont systématiquement libérés via `free()`.
 - **Processus :** Après chaque exécution du processus fils, le processus père exécute obligatoirement un appel synchrone à `waitpid` (via `syscall(61)`) pour intercepter le statut du fils, s'assurer de sa terminaison effective et libérer ses ressources auprès du noyau Linux.

d. Les contraintes d'interface et d'affichage (performance visuelle)

- **Description :** Le rafraîchissement à haute fréquence (pouvant descendre à moins d'une seconde, ex: 0.5s) ne doit pas provoquer de scintillement de l'écran (flickering) ni altérer l'historique du terminal de l'utilisateur.
- **Traduction dans le code :** Le programme est contraint d'utiliser le protocole des séquences d'échappement ANSI en mode brut plutôt que de lourdes bibliothèques graphiques. L'utilisation de `\033[?1049h` force le terminal à allouer une zone mémoire tampon isolée (l'écran alternatif). Le recours à `fflush(stdout)` après chaque bloc d'affichage garantit que les données ne restent pas bloquées dans le tampon du logiciel mais sont immédiatement poussées vers la carte graphique, assurant la fluidité visuelle de l'interface.

II. Architecture et Conception Technique (Le "Comment")

1. Architecture globale

Le programme s'organise autour d'une **boucle principale infinie** (while(1)) dans main.c
À chaque itération, le programme :

1. Exécute la commande demandée
2. Affiche le résultat à l'écran
3. Attend l'intervalle configuré en écoutant le clavier en parallèle

2. Logique algorithmique et structures de données

La structure principale utilisée est t_args, définie dans args.h, qui regroupe tous les paramètres du programme :

Champ	Type	Rôle
Cmd	char **	La commande à exécuter (tableau de mots)
interval	double	Intervalle de rafraîchissement (défaut : 2.0s)
highlight	int	Active le surlignage des différences (-d)
no_title	int	Masque l'en-tête (-t)
exit_error	int	Quitte si erreur (-e)
precise	int	Soustrait le temps d'exécution à l'attente (-p)
equexit	int	Quitte après N résultats identiques consécutifs (-q)
exec_direct	int	Exécution directe sans shell (-x)
Follow	int	Mode append sans effacer l'écran (-f)

Deux pointeurs **résultat** et **précédent** (chaînes dynamiques allouées avec **malloc/realloc**) permettent de comparer la sortie courante avec la sortie précédente, nécessaire pour le surlignage (-d) et les conditions de sortie (-g, -q).

3. Découpage modulaire (les fonctions clés)

Le programme est découpé en 9 modules avec des responsabilités bien séparées :

- **main()-main.c** : c'est le point d'entrée. Initialise la structure `t_args` via `parse_args()`, configure les signaux et le clavier, puis pilote la boucle principale. Gère aussi les conditions de sortie spéciales (-e, -g, -q).
- **parse_args()-args.c** : c'est l'analyse des arguments de la ligne de commande manuellement (sans `getopt`). Supporte également la variable d'environnement `WATCH_INTERVAL` comme valeur par défaut pour l'intervalle.
- **execute_cmd()-executor.c** : Ce module crée un pipe (syscall 22), fait un fork (syscall 57), puis dans le processus fils redirige `stdout/stderr` vers le pipe avec `dup2` (syscall 33) et lance la commande avec `execve` (syscall 59). Le père lit le résultat depuis le pipe en accumulant dynamiquement les données avec `realloc`, puis attend la fin du fils avec `waitpid` (syscall 61). En mode -x, la commande est lancée directement ; sinon elle est passée à `/bin/sh -c`.
- **display_screen()-display.c** : module qui efface l'écran affiche l'en-tête avec l'intervalle, la commande et l'heure courante, puis affiche le résultat. En mode -d, compare caractère par caractère avec la sortie précédente et surligne.
- **set_mode_clavier()/touche_appuyee()/lire_touche()-keyboard.c** : ici on configure le terminal en mode raw (syscall 16) pour lire les touches sans attendre la touche Entrée. `touche_appuyee()` utilise `select` (syscall 23) avec un timeout de 0 pour un test non-bloquant.
- **setup_signals()/precise_sleep()-utils.c** : `setup_signals()` installe un handler pour `SIGINT` (Ctrl+C) qui efface l'écran et appelle `exit()` proprement. `precise_sleep()` utilise `nanosleep()` pour une attente précise à la nanoseconde, utilisée dans les tranches de 0,1s de la boucle d'attente.
- **prendre_screenshot()-screenshot.c** : pour sauvegarder le contenu affiché dans un fichier texte nommé automatiquement avec la date et l'heure, en utilisant les syscalls `open` (2), `write` (1) et `close` (3) directement.
- **enlever_couleurs()-color.c** : filtre les séquences d'échappement ANSI du résultat pour le mode -C.
- **afficher_avec_wrap()-wrap.c** : tronque les lignes à la largeur du terminal (syscall 16) pour le mode -w et il y'a pas de retour à la ligne automatique.

III. Interfaces et maquettes

1. Interface de la ligne de commande

L'application se présente sous la forme d'un outil en ligne de commande exécutable depuis un terminal Unix. Son invocation nécessite le respect d'une syntaxe précise permettant de configurer la périodicité de l'affichage, d'activer le mode de détection visuelle des changements, et de définir la commande à surveiller.

L'utilisateur appelle l'outil en tapant le nom du programme exécutable, suivi de l'option de temporisation introduite par le drapeau dédié aux arguments temporels, de l'option d'inversion vidéo pour la mise en surbrillance, et enfin de la commande système cible spécifiée sous forme de chaîne de caractères entre guillemets. Dans sa configuration par défaut, l'outil applique un intervalle standard de deux secondes si aucun délai n'est explicitement fourni par l'opérateur. La variable temporelle est stockée sous forme de nombre à virgule flottante afin de

supporter des fréquences de rafraîchissement inférieures à la seconde pour un suivi de haute précision.

Par exemple, pour observer l'évolution de l'occupation de l'espace disque toutes les deux secondes, l'utilisateur saisit la commande d'appel en associant l'exécutable local à l'argument de l'intervalle et à la commande utilitaire de rapport d'espace disque entre guillemets.

2. Visuel du Terminal

Afin de proposer une expérience immersive conforme aux utilitaires système avancés, l'interface graphique de l'application utilise les capacités d'affichage plein écran offertes par les terminaux virtuels. Lors de son exécution, le programme masque l'invite de commande de l'utilisateur ainsi que l'historique des commandes passées pour ouvrir un espace de rendu dédié et statique, éliminant ainsi tout effet de défilement textuel vertical.

L'interface visuelle finale se structure en deux zones horizontales distinctes :

- **L'en-tête** : positionné sur la toute première ligne du terminal, affiche de manière fixe à l'extrême gauche la mention de la période de rafraîchissement en secondes accompagnée du libellé de l'application. À l'extrême droite de cette même ligne, un horodatage complet et dynamique extrait l'heure système en temps réel et met à jour les secondes, les minutes, les heures et la date à chaque cycle d'exécution, selon le format standardisé combinant l'année, le mois, le jour et l'heure courante. Une ligne vide sépare cet en-tête du reste de l'écran pour aérer l'interface.
- **Le corps de l'affichage** : occupe tout le reste de l'écran et restitue fidèlement le flux textuel renvoyé par la commande système exécutée. Lorsque l'option de surbrillance est activée par l'utilisateur, l'interface applique instantanément un traitement visuel différentiel : chaque caractère ou valeur numérique qui diverge par rapport à la capture effectuée lors du cycle précédent est affiché en vidéo inversée. Ce contraste marqué entre la couleur du texte et celle du fond permet à l'opérateur de détecter immédiatement la moindre variation de données d'une copie à l'autre.

3. Conception technique du module d'affichage

La gestion globale de l'interface et de la console est entièrement centralisée au sein d'un module d'affichage isolé. La conception de ce module repose sur l'envoi de chaînes d'instructions spécifiques, appelées séquences d'échappement ANSI, directement vers le flux de sortie standard. Ces séquences permettent de piloter le comportement matériel du terminal sans dépendre de bibliothèques graphiques tierces lourdes.

Le module s'articule autour de quatre fonctions techniques majeures :

- **La fonction d'initialisation** : prépare l'environnement système dès l'ouverture du programme. Elle transmet au terminal l'ordre de basculer vers le tampon de l'écran alternatif, ce qui isole l'affichage de l'application de l'historique de la console, puis elle désactive le curseur clignotant afin d'éviter tout scintillement désagréable lors des phases de réécriture. Un vidage systématique de la mémoire tampon de sortie garantit l'application immédiate de ces paramètres.

- **La fonction de gestion** : de l'en-tête prend en charge le nettoyage et la mise à jour de la zone supérieure à chaque nouveau cycle. Elle commence par effacer l'intégralité des données de l'écran et repositionne le curseur virtuel au point d'origine, en haut à gauche. Elle sollicite ensuite les fonctionnalités de gestion du temps de la bibliothèque standard pour capturer l'instant présent, le formater proprement en chaîne de caractères, et l'injecter dans la structure d'affichage avec une précision d'une décimale pour l'intervalle de temps.
- **La fonction de traitement du contenu** : est chargée de l'analyse comparative et du rendu du texte de la commande. Elle parcourt la table des caractères de la sortie actuelle et la confronte à celle de la sortie précédente. Si le mode de surbrillance est activé et qu'une différence est relevée sur un octet précis, la fonction encapsule ce caractère entre une instruction d'inversion des couleurs de fond et de premier plan et une instruction de réinitialisation. Les caractères identiques sont quant à eux envoyés directement au gestionnaire de flux standard.
- **La fonction de nettoyage** : assure la fermeture propre de l'application et la restitution des ressources. Principalement appelée lors de l'interception d'un signal d'interruption volontaire de l'utilisateur, elle effectue les actions inverses de la phase d'initialisation en réactivant le curseur textuel traditionnel et en fermant l'écran alternatif. Ce mécanisme garantit que le terminal de l'utilisateur retrouve son état et son comportement d'origine dès la fermeture du programme.

IV. Plan des tests de validation

Afin de garantir la robustesse de notre commande mywatch et de valider sa conformité vis-à-vis des exigences du projet, une stratégie de test rigoureuse a été planifiée pour cerner les limites fonctionnelles du programme. Cette démarche de validation s'articule autour de cinq phases méthodologiques bien précises. La première phase évalue l'analyseur d'arguments sous ses options par défaut afin de vérifier le comportement standard de la commande lors d'une exécution de base. La deuxième phase examine la modification des valeurs des arguments pour s'assurer que le système prend correctement en compte les variables personnalisées saisies par l'utilisateur. La troisième phase est dédiée à la vérification de la validité des arguments ; elle certifie l'existence des options appelées et contrôle la conformité du nouvel intervalle de temps, déclenchant l'émission d'un message d'erreur systématique si une anomalie est détectée. La quatrième phase valide la sensibilité aux touches du clavier après le lancement de la commande, confirmant par exemple que l'interception de la touche « q » entraîne l'interruption immédiate et propre de l'exécution. Enfin, la cinquième phase traite le cas d'absence de commande, garantissant qu'un message d'erreur explicite est généré pour bloquer l'application si l'utilisateur omet de spécifier le programme à surveiller.

CONCLUSION

La réalisation de la commande **mywatch** constitue une opportunité majeure de confronter les notions théoriques de notre unité d'enseignement aux réalités de la programmation système de bas niveau en langage C. En parvenant à nous affranchir des abstractions de haut niveau pour concevoir manuellement notre analyseur d'arguments et en communiquant directement avec le noyau Linux via les appels système, nous avons développé une solution robuste, modulaire et conforme au cahier des charges. L'intégration réussie du contrôle clavier en temps réel par syscall, des alertes sonores et de la gestion de l'affichage valide la viabilité technique de notre clone. Au-delà des compétences acquises en gestion des processus et des flux, ce projet a renforcé notre cohésion d'équipe sera l'élément indispensable qui permettra de livrer un outil fonctionnel et rigoureux.